IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Digital Object Identifier

A Mechanically Verified Translation of $\mathrm{MTL}_{0,\infty}$ into Timed Büchi Automata

ELIE FARES[1], JEAN-PAUL BODEVEIX[2], HUSSAIN AL-AQRABI[1] AND AZAR SALAMI[1]
[1]Higher Colleges of Technology, Sharjah, United Arab Emirates (e-mail: efares@hct.ac.ae, halaqrabi@hct.ac.ae, asalami@hct.ac.ae)
[2]IRIT, Université de Toulouse, France (e-mail: bodeveix@irit.fr)

Corresponding author: Elie Fares (e-mail: efares@hct.ac.ae).

ABSTRACT Translation from real-time temporal specifications to timed automata supports automated analysis, but depends on preserving the specified timed-word language. We present a Rocq-checked soundness-and-completeness development for the specified pointwise dense-time translation from Metric Temporal Logic $(\mathrm{MTL})_{0,\infty}$ to timed Büchi automata (TBAs), via clocked Linear Temporal Logic (LTL). Earlier work established mathematical language-preservation results for this fragment; to the best of our knowledge, no published machine-checked soundness-and-completeness development covers the particular construction and semantic model formalized here. Rocq checks all eight strict and non-strict primitive cases. The proof handles shared clocks and overlapping activations by separating soundness for every clock-consistent reset trace from completeness via one canonical reset trace and residual obligations. It also checks relaxation and reset completion, yielding a compositional theorem under an explicit all-words language-equivalence contract for the propositional Büchi backend. For an alarm-response instance, we prove that a manually transcribed backend record satisfies this contract and prove both an accepted overlapping-alarm trace and a violating trace. The theorem uses real-valued bounds, well-formed core formulas, strictly increasing divergent timestamps, and event-guarded TBAs with existential initial clock valuations; the Spot graph-to-record correspondence is outside the proof. On 31 benchmark configurations and 100 random formulas, we measure the automata produced by extracted code, check language inclusions against CASAAL with TChecker, compare the semantics with the automaton on 26,200 random timed words, 25,400 of them also against CASAAL, and test the backend contract with an independent translator.

INDEX TERMS Metric temporal logic, timed Büchi automata, Rocq, formal verification, clocked LTL, semantic preservation.

I. INTRODUCTION

Reliable translation from real-time Metric Temporal Logic (MTL) to timed Büchi automata (TBAs) supports model checking and controller synthesis. Its correctness is the semantic link between a temporal specification and the automaton analyzed by a downstream tool. A bounded-response requirement, for example, is

$$\Box(req \Rightarrow \Diamond_{\leq d}\, ack).$$

Overlapping requests can leave several response obligations outstanding while sharing the clock assigned to one primitive timed formula. Table 2 illustrates why the reset semantics need a correctness argument.

The mathematical construction studied here has substantial predecessors. Bulychev et al. establish exact timed-automaton language results for safety and co-safety fragments. Li et al. establish full-fragment $\mathrm{MTL}_{0,\infty}$-to-TBA language equality and show that their best-choice reset policy preserves language [1], [2]. Their work already contains the central timed-subformula clocking and reset architecture. Our research result is a machine-checked proof for the specified construction. Table 8 compares them.

Mathematical proofs and machine-checked proofs provide different assurance. A proof assistant requires the syntax, timed-word semantics, clock updates, invariants, quantifiers, and proof dependencies to be represented in a kernel-checkable development. Here that formalization must also reconcile propositions of clocked Linear Temporal Logic (LTL) with genuine clock valuations. Related proof-assistant work addresses neighboring problems: Xu and Miao present a Metric Interval Temporal Logic (MITL)-to-fair-timed-automaton construction and discuss its implementa-

tion in the Prototype Verification System (PVS); Roohi and Viswanathan mechanize results for MITL decision procedures in PVS; Schimpf et al. verify an LTL-to-Büchi construction in Isabelle's higher-order logic (HOL); Wang et al. verify a Metric Linear Temporal Logic (MLTL)-to-regular-expression transformation in Isabelle/HOL; and Chattopadhyay and Mamouras mechanize online monitoring for past-time MTL under discrete-time quantitative semantics [3]–[7]. To the best of our knowledge, no previous published machine-checked soundness-and-completeness development has been identified for this particular pointwise $\mathrm{MTL}_{0,\infty}$-to-TBA construction and formal semantic model, including its clocked-LTL encoding, relaxation, and reset-completion steps. This scoped claim recognizes the existing proof-assistant work on related logics and constructions.

Several interacting proof obligations make the mechanization nontrivial. Soundness must hold for every clock-consistent reset trace; completeness must construct one trace that supports every dynamic activation of a shared, formula-keyed clock. Independent witnesses per activation cannot establish this single-trace property. The proof maintains residual obligations for upper- and lower-bound Until and Release, and uses occurrence-indexed structural induction so recursion through a syntax path retains the clock identity of an identical primitive formula. Strict and non-strict boundaries require distinct inequalities. In addition, the propositional backend can assign clock atoms independently of actual clock valuations, so relaxation and reset completion must preserve the precise implication directions before the automaton theorem can be composed.

The principal contributions are:

- Rocq-checked soundness and completeness of the clocked-LTL encoding, including all eight strict and non-strict hatted primitive cases and correctness at every syntactic occurrence;
- a canonical completeness construction that handles overlapping activations with one reset trace for each formula-keyed clock, together with policy-independent soundness for arbitrary consistent reset traces;
- checked relaxation and reset completion, composed with an explicit all-propositional-words backend contract in MTL_to_TBA_correct_with; and
- a concrete alarm-response instance whose manually transcribed Rocq backend satisfies the contract, with proved accepted-overlap and rejected-violation traces, and an inspectable proof and assumptions artifact; and
- an evaluation of the automaton produced by the verified construction, before any optimization: sizes against CASAAL, language-inclusion checks with TChecker, and tests of the backend contract (Section VI).

The result is relative to the formal model defined here: real-valued bounds, well-formed core formulas, strictly increasing divergent timestamps, event-guarded TBAs without location invariants, and existential nonnegative initial clocks. This differs from Li et al.'s integer-bound, nondecreasing-time, zero-initial semantics and is not a direct refinement of CASAAL's input/output semantics. The TBA theorem composes through an explicit backend-equivalence premise; for the alarm instance Rocq checks a manually transcribed backend record, not Spot or the graph-import path. Zero-bound preprocessing is described but not verified, and the development proves a semantic theorem; the evaluated tool runs code extracted from its definitions, and extraction, parsing, and printing are outside the proof. Section V-F summarizes the theorem dependencies and assumptions. Optimization of the resulting automata and their export to UPPAAL are outside the scope of this article; they are left to follow-up work.

Section II defines the semantics and target model; Sections III–IV present the encoding and automaton construction; Section V develops the proof and includes the alarm-response instance in Section V-G; Section VI evaluates the construction, Section VII describes the artifact and reproduction procedure, and Section VIII discusses related work before the conclusion .

II. LOGIC AND SEMANTIC MODEL

A. POINTWISE $\mathrm{MTL}_{0,\infty}$

Let Σ be a finite set of actions. Formulas are in negation normal form and use Boolean operators, next, untimed Until and Release, and timed Until and Release whose intervals have one endpoint at zero or infinity. We write a bound as $\sim\ d$, with $\sim \in \{<, \leq, >, \geq\}$. The logic also contains the hatted operators $\widehat{\mathsf{U}}_{\sim d}$ and $\widehat{\mathsf{R}}_{\sim d}$. In a hatted operator the current position is skipped by the logical interval, while its timestamp remains the timing reference. We use $\widehat{\Box}_{\sim d}\varphi$ as shorthand for $\bot\ \widehat{\mathsf{R}}_{\sim d}\ \varphi$; it requires φ at each strictly future position whose elapsed time from the current position satisfies the bound.

A timed word is $w = (a_i, t_i)_{i\in\mathbb{N}}$, where $a_i \in \Sigma$, $0 \leq t_0 < t_1 < \cdots$, and timestamps diverge. We write $\mathsf{elap}_w(i,j) = t_j - t_i$. For Until, $w, i \models_{\mathrm{MTL}} p\ \mathsf{U}_{\sim d}\ q$ holds when some $j \geq i$ satisfies q and $t_j - t_i \sim d$, with p at each position in $[i,j)$. For hatted Until, the witness is $j > i$ and p is required on (i,j). Release is the dual universal condition: at every position in the bounded region, either q holds or p held earlier in the corresponding interval. Hatted Release excludes the current position from that region. We use the standard pointwise semantics [8].

For precision, the four bounded clauses used below are:

$$w,i \models_{\mathrm{MTL}} p\ \mathsf{U}_{\sim d}\ q \iff \exists j \geq i.\ w,j \models_{\mathrm{MTL}} q \\ \wedge\ t_j - t_i \sim d \\ \wedge\ \forall k \in [i,j).\ w,k \models_{\mathrm{MTL}} p,$$

$$w,i \models_{\mathrm{MTL}} p\ \widehat{\mathsf{U}}_{\sim d}\ q \iff \exists j > i.\ w,j \models_{\mathrm{MTL}} q \\ \wedge\ t_j - t_i \sim d \\ \wedge\ \forall k \in (i,j).\ w,k \models_{\mathrm{MTL}} p,$$

$$w,i \models_{\mathrm{MTL}} p\ \mathsf{R}_{\sim d}\ q \iff \forall j \geq i. \\ \Big(t_j - t_i \sim d \Rightarrow \\ \Big(w,j \models_{\mathrm{MTL}} q \\ \vee\ \exists k \in [i,j).\ w,k \models_{\mathrm{MTL}} p\Big)\Big),$$

$$w,i \models_{\mathrm{MTL}} p\ \widehat{\mathsf{R}}_{\sim d}\ q \iff \forall j > i. \\ \Big(t_j - t_i \sim d \Rightarrow \\ \Big(w,j \models_{\mathrm{MTL}} q \\ \vee\ \exists k \in (i,j).\ w,k \models_{\mathrm{MTL}} p\Big)\Big).$$

The index intervals select event positions, whereas $t_j - t_i$ measures elapsed physical time. Thus, when a bound falls between two events, no event occurs exactly at that bound, and interval membership is still evaluated at the event positions. The strict increase assumption excludes simultaneous events; divergence is used in the lower-bounded Until fairness argument.

The Rocq development represents action names by natural numbers rather than a parameterized finite type, and timed words may use any natural-number name. Each formula mentions finitely many names, so its semantics and generated labels distinguish actions only by equality or inequality with those names. Equivalently, the construction operates over the finite quotient induced by those action tests; this is the finite alphabet used in the mathematical presentation.

The well-formedness condition requires $d > 0$ for every lower bound and strict upper bound, and $d \geq 0$ for a non-strict upper bound. Ordinary timed operators are definitions in the mechanized core. For upper bounds $\precsim \in \{\leq, <\}$ and lower bounds $\succsim \in \{\geq, >\}$, they are:

$$p\ \mathsf{U}_{\precsim d}\ q \equiv q \vee (p \wedge (p\ \widehat{\mathsf{U}}_{\precsim d}\ q)), \tag{1}$$

$$p\ \mathsf{U}_{\succsim d}\ q \equiv p \wedge (p\ \widehat{\mathsf{U}}_{\succsim d}\ q), \tag{2}$$

$$p\ \mathsf{R}_{\precsim d}\ q \equiv q \wedge (p \vee (p\ \widehat{\mathsf{R}}_{\precsim d}\ q)), \tag{3}$$

$$p\ \mathsf{R}_{\succsim d}\ q \equiv p \vee (p\ \widehat{\mathsf{R}}_{\succsim d}\ q). \tag{4}$$

These definitions separate the current-position case from the strictly future obligation. Their pointwise semantic compatibility, including the stated bound hypotheses, is proved in Rocq:

Theorem II.1 (Semantic compatibility of the derived timed operators)**.** *Under the well-formedness conditions above, Eqs.* (1)– (4) *agree with the standard pointwise semantics for all strict and non-strict upper and lower bounds.*

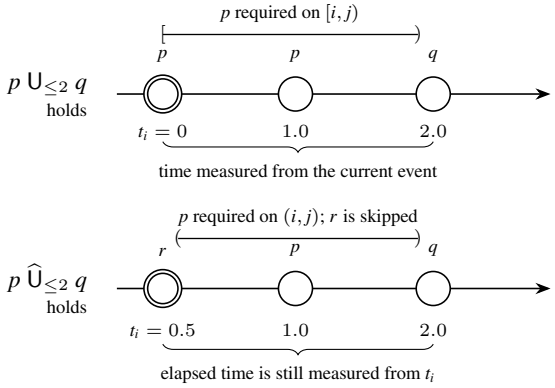


FIGURE 1. Ordinary and hatted Until. The double circle is the timing reference. The hatted operator skips the current event logically but measures from its timestamp.

The identities follow by separating the current-position case from a strictly future endpoint. For upper Until, a witness at i contributes q; every later witness requires p at i and leaves the hatted obligation. A lower bound with $d > 0$ excludes i, so p is required now and the remaining witness is hatted. The Release identities are the dual case split: an upper-bound region includes i and therefore requires q there, while a lower-bound region excludes i. Strict upper bounds also require $d > 0$, avoiding the degenerate < 0 case. The eight corresponding Rocq lemmas are mapped to their source modules and assumptions in Table 5. Thus only the eight hatted timed constructors require primitive translation clauses. If a source formula has a zero lower bound, strict increase of timestamps makes every strictly future elapsed time positive. At the hatted level, $\widehat{\mathsf{U}}_{\geq 0}$ and $\widehat{\mathsf{U}}_{>0}$ therefore reduce to $\mathsf{X}(p\ \mathsf{U}\ q)$, while $\widehat{\mathsf{R}}_{\geq 0}$ and $\widehat{\mathsf{R}}_{>0}$ reduce to $\mathsf{X}(p\ \mathsf{R}\ q)$. For ordinary operators, the current-position case must also be simplified: the ≥ 0 cases are the corresponding untimed Until and Release operators, and the > 0 cases reduce to $p \wedge \mathsf{X}(p \mathsf{U} q)$ and $p \vee \mathsf{X}(p\ \mathsf{R}\ q)$, respectively. These reductions rely on the paper's strictly increasing event times and must be applied before encoding a source formula as a Rocq core formula; the preprocessing itself is not mechanized. A zero strict upper bound has an empty interval: Until reduces to $\bot$ and Release to $\top$; strict upper-bound constructors in the core consequently require $d > 0$. For the non-strict upper bound zero, the hatted future operators reduce to $\widehat{\mathsf{U}}_{\leq 0} \equiv \bot$ and $\widehat{\mathsf{R}}_{\leq 0} \equiv \top$, because all strictly future events have positive elapsed time. The derived ordinary operators therefore both reduce to their right operand: $p \mathsf{U}_{\leq 0} q \equiv q$ and $p \mathsf{R}_{\leq 0} q \equiv q$. These reductions make the boundary explicit: the mechanized domain excludes zero lower bounds and zero strict upper bounds, but includes non-strict upper bound zero.

This distinction is useful for sporadicity: if consecutive occurrences of e must be separated by at least θ, one may write $\Box(e \Rightarrow \widehat{\Box}_{<\theta} \neg e)$. The hatted form retains the triggering event as the time origin; ordinary bounded temporal operators do not express this particular reference shift directly. Related

pointwise MTL constructions and the CASAAL translation are discussed in prior work [2], [9]–[11].

B. TIMED BÜCHI AUTOMATA

The verified target is an event-guarded TBA $\mathcal{A} = (L, \ell_0, X, E, Acc)$. Here L is a finite set of locations, ℓ_0 the initial location, X a finite set of clocks, $E \subseteq L \times \mathsf{Lab} \times \mathsf{Guard} \times 2^X \times L$ is the set of action-labelled transitions with guards and reset sets, and $Acc \subseteq L$ the Büchi set. A run is a sequence $(\ell_i, v_i)_{i\geq 0}$ starting at ℓ_0; it follows one transition per event and visits Acc infinitely often. If the selected edge at event i is $(\ell_i, \alpha_i, g_i, Z_i, \ell_{i+1})$, then the event action must satisfy α_i and the valuation v_i must satisfy every guard in g_i. There are no location invariants in this model: timing constraints are checked at the events of the timed word. This is the TBA record represented in Rocq; the theorem does not cover a richer invariant-bearing automaton model without an additional encoding.

Let v_i be the clock valuation tested at event i and $\Delta_i = t_{i+1} - t_i$. A transition at event i tests v_i before its reset decision; the next valuation is

$$v_{i+1}(x) = \begin{cases} \Delta_i, & x \in Z_i, \\ v_i(x) + \Delta_i, & \text{otherwise.} \end{cases}$$

The initial nonnegative valuation v_0 is existentially quantified, as in the Rocq clock-consistent-extension semantics; standard zero initialization is not assumed.

III. CLOCKED-LTL TRANSLATION

A. SHARED CLOCK IDENTITIES

The translation is structural. It maps Boolean and untimed temporal operators homomorphically and maps each primitive timed subformula to an LTL formula over an extended alphabet. For each action a, the extended alphabet contains distinct action atoms $\mathsf{LAct}(a)$ and $\mathsf{LNAct}(a)$, as well as clock-related atoms: clock comparisons $x \sim d$, a reset atom $\mathsf{rst}(x)$, and a preservation atom $\mathsf{unch}(x)$. Under timed-word interpretation, the action atoms mean that the current action is a or is different from a, respectively. The atom $\mathsf{LNAct}(a)$ and the negated literal of $\mathsf{LAct}(a)$ have the same timed-word interpretation, but remain distinct propositions to the unconstrained LTL backend. The reset and preservation atoms describe the clock update at the current event. Clock values are read before the event's reset, matching the guard semantics of a timed automaton.

For formula f, let $Clock(f)$ be its finite set of distinct primitive hatted timed subformulas. The clock associated with $F \in Clock(f)$ is $x = F$ itself. Clock identity uses structural equality on the Rocq MTL syntax tree, with bounds compared by equality in $\mathbb{R}$. Constructor, operands, comparison, and bound all participate: identical formula values at different paths share a clock, while semantic equivalence alone does not. No simplification-based merging is used. The main result proves that this clock also serves all dynamic activations of its subformula. The clock count is therefore bounded by the number of distinct primitive timed subformulas, which is at most the number of timed occurrences in f. If structural subformulas are represented as shared nodes, the translation can share their LTL encodings as well; this does not remove the usual worst-case growth of an LTL-to-Büchi construction.

This clock-sharing policy is distinct from automaton-state minimization. The reset trace is chosen for each formula identity as a whole. In particular, the proof cannot justify the clock bound by assigning a fresh clock to each activation and subsequently identifying those clocks: it establishes directly that a single trace of reset decisions for the formula is sufficient.

TABLE 1. Clock comparisons used by each hatted primitive.

Primitive role	Non-strict bound	Strict bound
$\widehat{\mathsf{U}}_{\precsim d}$ endpoint	$x \leq d$	$x < d$
$\widehat{\mathsf{U}}_{\succsim d}$ threshold	$x \geq d$	$x > d$
$\widehat{\mathsf{R}}_{\precsim d}$ window end	$x > d$	$x \geq d$
$\widehat{\mathsf{R}}_{\succsim d}$ pre-threshold	$x < d$	$x \leq d$

B. EIGHT PRIMITIVE CLAUSES

Let F be a primitive hatted formula at path π, with operands p, q. Write $x = \mathsf{clock_of}(F)$, $P = \mathcal{T}_{\pi 0}(p)$, $Q = \mathcal{T}_{\pi 1}(q)$, and define $\alpha \mathsf{W} \beta = (\alpha \mathsf{U} \beta) \vee \mathsf{G}\alpha$. For upper comparisons $\precsim \in \{\leq, <\}$ and lower comparisons $\succsim \in \{\geq, >\}$, complements are $\leq^{\mathrm{c}} = >$, $<^{\mathrm{c}} = \geq$, $\geq^{\mathrm{c}} = <$, and $>^{\mathrm{c}} = \leq$. The four clause shapes, each instantiated for its strict and non-strict bound, give all eight primitive cases:

$$\mathcal{T}_\pi(p\, \widehat{\mathsf{U}}_{\precsim d}\, q) = \mathsf{X}\big(((x \precsim d) \wedge \mathsf{unch}(x) \wedge P) \mathsf{U} ((x \precsim d) \wedge Q)\big), \tag{5}$$

$$\mathcal{T}_\pi(p\, \widehat{\mathsf{U}}_{\succsim d}\, q) = \mathsf{rst}(x) \wedge \mathsf{X}\big((P \mathsf{U} ((x \succsim d) \wedge (P \mathsf{U} Q))) \vee (\mathsf{G}P \wedge \mathsf{GF}Q)\big), \tag{6}$$

$$\mathcal{T}_\pi(p\, \widehat{\mathsf{R}}_{\precsim d}\, q) = \mathsf{rst}(x) \wedge \mathsf{X}\big(Q \mathsf{W} ((x \precsim^{\mathrm{c}} d) \vee (P \wedge Q))\big), \tag{7}$$

$$\mathcal{T}_\pi(p\, \widehat{\mathsf{R}}_{\succsim d}\, q) = \mathsf{X}\big(((x \succsim^{\mathrm{c}} d) \wedge \mathsf{unch}(x)) \mathsf{W} ((P \mathsf{R} Q) \vee ((x \succsim^{\mathrm{c}} d) \wedge P))\big). \tag{8}$$

The outer X implements the hatted shift; $\mathsf{rst}(x)$ and $\mathsf{unch}(x)$ encode the restart and preservation choices. Until uses strong Until for a bounded witness and weak Until for the Release window. Lower-bounded Until also has a fairness branch for an obligation whose threshold is delayed indefinitely. Every clock atom occurs positively, which is the property used by relaxation in Section IV-B. The eight local proof arguments establish the precise strict and non-strict behavior in Section V-A.

TABLE 2. A finite prefix illustrating overlapping response obligations. The deadline is five time units; the acknowledgment at time six is late for the request at time zero even though it is age two from the later request.

Event i	Action	Time t_i	Oldest age	Reset age
0	*req*	0	0	0
1	*req*	4	4	4
2	*ack*	6	6	2

C. SHARED CLOCKS UNDER OVERLAPPING ACTIVATIONS

Hatted operators preserve the triggering timestamp while moving logical evaluation to the next event. For example, $\Box(e \Rightarrow \widehat{\Box}_{<\theta}\neg e)$ measures an exclusion window from the triggering event. The leading next in an ordinary operator would shift that reference, while omitting it also constrains the triggering event. Figure 2 contrasts these readings.

The proof assigns one clock to each structurally distinct primitive timed formula, even when that formula is activated repeatedly. The finite prefix in Table 2 illustrates why completeness must account for all pending obligations: resetting a single clock at the later request can make an expired earlier deadline appear to be met. The table illustrates a generic reset risk and makes no claim about an error in prior tools. This is only motivation. The semantic proof below constructs one trace per formula-keyed clock and establishes the residual invariants needed when several activations are pending at once.

IV. FROM CLOCKED LTL TO A TBA

A. LTL-TO-BÜCHI CONTRACT

The backend treats extended atoms as independent propositions. It returns a propositional Büchi automaton (PBA) whose language is exactly that of the clocked-LTL formula. The theorem is stated for any such backend, deterministic or not.

Assumption IV.1 (Backend correctness)**.** *For each formula F and each propositional word s over its atoms,* $\mathsf{PBA_accepts}(A, s)$ *iff* $s, 0 \models F$*, where A is the backend automaton for F.*

This contract ranges over all propositional words, including assignments that do not correspond to any clock-consistent timed word. That stronger scope is used by relaxation.

B. RELAXATION AND RESET COMPLETION

The propositional backend regards every extended atom as independent. Its correctness contract therefore applies to all Boolean words, including assignments that cannot arise from any clock-consistent timed trace. Every clock-related atom in the translation occurs positively. Relaxation acts on each backend transition cube by removing negative literals over clock comparisons and reset/preservation markers, removing irrelevant clock-related literals, and discarding contradictory cubes. Action literals and positive relevant clock literals are retained. This weakens the cubes without asking the backend to understand clocks.

Reset completion converts each relaxed cube to a timed transition. A preservation clock is reset unless the cube contains $\mathsf{unch}(x)$; a restart clock is reset exactly when the cube contains $\mathsf{rst}(x)$. The markers are then removed, clock comparisons become guards, and action literals become labels. The two clock classes move in opposite directions, so the proof treats them separately.

Lemma IV.2 (Reset completion)**.** *Let an accepting run of the relaxed backend be accompanied by a clock-consistent extension ρ of its timed word. Reset completion defines a clock-consistent extension ρ' with the same timed word and the same initial clock valuation. The same state sequence is an accepting run of the completed TBA on ρ'.*

Proof. Fix a selected backend cube at position i. For a preservation clock x, if the cube contains $\mathsf{unch}(x)$ then both traces preserve x. Otherwise the completed TBA resets x, while ρ may either reset it or preserve it. Thus the completed reset set contains the original reset decision for x. Induction on positions using the clock recurrence gives $\mathsf{val}_{\rho'}(i,x) \le \mathsf{val}_{\rho}(i,x)$. These clocks occur only in upper tests ($x \le d$ or $x < d$), which remain true when the value decreases.

For a restart clock x, completion resets it exactly when the cube contains $\mathsf{rst}(x)$. A true reset marker requires that ρ resets x; without the marker completion preserves it, whether or not ρ resets it. The completed reset set is therefore contained in the original one, and the same induction gives $\mathsf{val}_{\rho'}(i,x) \ge \mathsf{val}_{\rho}(i,x)$. These clocks occur only in lower tests ($x \ge d$, $x > d$, or their equivalent window-ending comparisons), which remain true when the value increases. The clock recurrence holds by the construction of ρ'. State transitions, event labels, guards that remain true, and the Büchi accepting set are unchanged, so the run remains accepting. Rocq proves this result as complete_complete. □

The role of propositional valuations, relaxation, and reset completion in both directions is proved once, as part of the end-to-end composition in Theorem V.4. A propositional model alone is not a clock-consistent timed trace.

C. WORKED EXAMPLE: BOUNDED EVENTUALITY WITH AN EXCLUSION WINDOW

Let

$$f = (\mathsf{F}_{\le 5} p) \wedge (\mathsf{G}_{<2} \neg p).$$

It requires p to be false at the origin and until age two, then true by age five. Figure 3 shows a hand-simplified equivalent one-clock TBA, not the syntax-directed output, which assigns separate clocks to the two distinct timed subformulas. The steps below show the clocked-LTL translation and timed reinterpretation. This is a construction illustration: the saved Spot graph and its two-clock TBA transcription are not covered by the proved Rocq backend contract. The mechanically verified concrete instance is the alarm-response example in Section V-G.

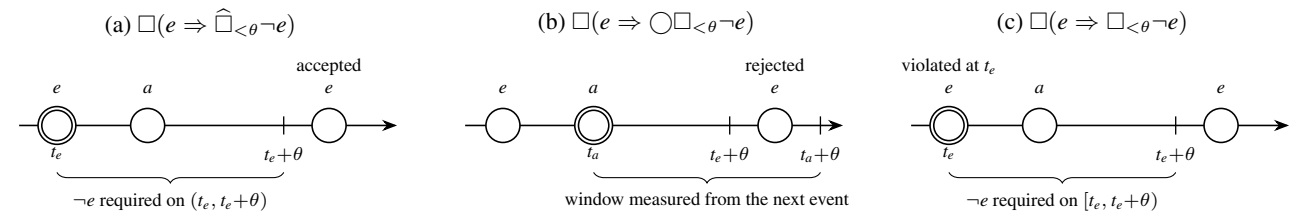


FIGURE 2. **The hatted sporadicity property and two ordinary-operator substitutions. A leading next shifts the timing reference to the following event; omitting it includes the triggering event in the window.**

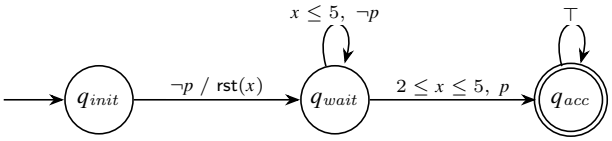


FIGURE 3. **Hand-simplified equivalent event-guarded TBA for f. The initial transition requires $\neg p$ and resets x; in the waiting location, $x \le 5$, and acceptance also requires $x \ge 2$. Thus p first holds at age in $[2, 5]$.**

Step 1: The formula is in negation normal form. Expanding the derived operators and applying the timed-operator identities gives

$$\begin{aligned} f &= (\top \,\mathsf{U}_{\le 5}\, p) \wedge (\bot \,\mathsf{R}_{<2}\, \neg p) \\ &= \big(p \vee (\top \wedge \top\, \widehat{\mathsf{U}}_{\le 5}\, p)\big) \\ &\qquad \wedge \big(\neg p \wedge (\bot \vee \bot\, \widehat{\mathsf{R}}_{<2}\, \neg p)\big). \end{aligned}$$

These current-position cases follow Eqs. (1) and (3).

Step 2: The two distinct primitive timed subformulas receive clocks x and y, respectively. Their clocked-LTL translation is

$$\begin{aligned} \Phi_U &= p \vee \Big(\top \wedge \mathsf{X}\Big(((x \le 5) \wedge \mathsf{unch}(x) \wedge \top)\mathsf{U} \\ &\qquad ((x \le 5) \wedge p)\Big)\Big), \\ \Phi_R &= \neg p \wedge \big(\bot \vee (\mathsf{rst}(y) \wedge \\ &\qquad \mathsf{X}(\neg p\ \mathsf{W}\ ((y \ge 2) \vee (\bot \wedge \neg p))))\big), \\ \mathcal{T}(f) &= \Phi_U \wedge \Phi_R. \end{aligned}$$

The Until preserves x; the Release test $y \ge 2$ complements the source bound < 2, allowing p from age two.

Step 3: The saved Spot 2.16 graph for $\mathcal{T}(f)$ is redrawn on the left of Fig. 4. Clock guards, reset and preservation markers, and action atoms are independent propositions to the backend. In particular, $p = \mathsf{LAct}(p)$ and $\neg p = \mathsf{LNAct}(p)$ are distinct atoms, while $!p$ is the negated literal of $\mathsf{LAct}(p)$; a backend cube may contain both p and $\neg p$. Timed reinterpretation imposes action consistency, so some backend edges have no realizable timed event.

Step 4: Relaxation removes negative literals over extended clock atoms. Reset completion resets the preservation clock x unless $\mathsf{unch}(x)$ is present; the restart clock y is reset exactly when $\mathsf{rst}(y)$ is present. The right side of Fig. 4 shows the same state graph after these operations. Edge identifiers replace dense labels; Table 3 gives each Spot cube and the corresponding event condition, surviving clock guard, and completed reset set. The table retains unsatisfiable action cubes to make the propositional-to-timed boundary visible; removing them would be a subsequent simplification.

.

V. MECHANIZED CORRECTNESS

For a formula f, an extended word ρ augments the timed word with a nonnegative value $\mathsf{val}_\rho(i,x)$ and a reset decision $\mathsf{reset}_\rho(i,x)$ for every formula-keyed clock $x \in \mathit{Clock}(f)$ at each position i. It is clock-consistent when, with $\Delta_i = t_{i+1} - t_i$,

$$\mathsf{val}_\rho(i+1,x) = \begin{cases} \Delta_i, & \mathsf{reset}_\rho(i,x), \\ \mathsf{val}_\rho(i,x) + \Delta_i, & \text{otherwise.} \end{cases} \tag{9}$$

The value at i is tested before the reset decision at i. The relation $\mathsf{same_base}(\rho, w)$ means that projection of ρ onto actions and timestamps gives w.

A. SOUNDNESS FOR ARBITRARY RESET TRACES

The soundness direction does not select or assume the canonical policy. For each primitive formula F at path π, the corresponding Rocq lemma `sound_`$\star$ turns $\rho, i \models_{\mathrm{LTL}} \mathcal{T}_\pi(F)$ into $\mathsf{ew_base}(\rho), i \models_{\mathrm{MTL}} F$, assuming clock consistency and induction hypotheses for its operands. The four proof families below each cover both the strict and non-strict comparison.

Clock consistency gives two useful bounds. If a clock is preserved on $[i,j)$, then

$$\mathsf{val}_\rho(j,x) = \mathsf{val}_\rho(i,x) + \mathsf{elap}_w(i,j).$$

Since clock values are nonnegative, preservation implies $\mathsf{val}_\rho(j,x) \ge \mathsf{elap}_w(i,j)$. If it is reset at i, then for every $j \ge i+1$,

$$\mathsf{val}_\rho(j,x) \le \mathsf{elap}_w(i,j),$$

with equality when it is not reset again before j. The upper-guard implication follows from the first inequality; the lower-guard implication follows from the second. Both follow by induction from Eq. (9) and nonnegative delays.

Upper-bounded hatted Until. The outer X moves to the first strictly future event. The strong Until then supplies a witness $j > i$ satisfying Q, with P at every intermediate event and $\mathsf{unch}(x)$ throughout that prefix. Regardless of whether the

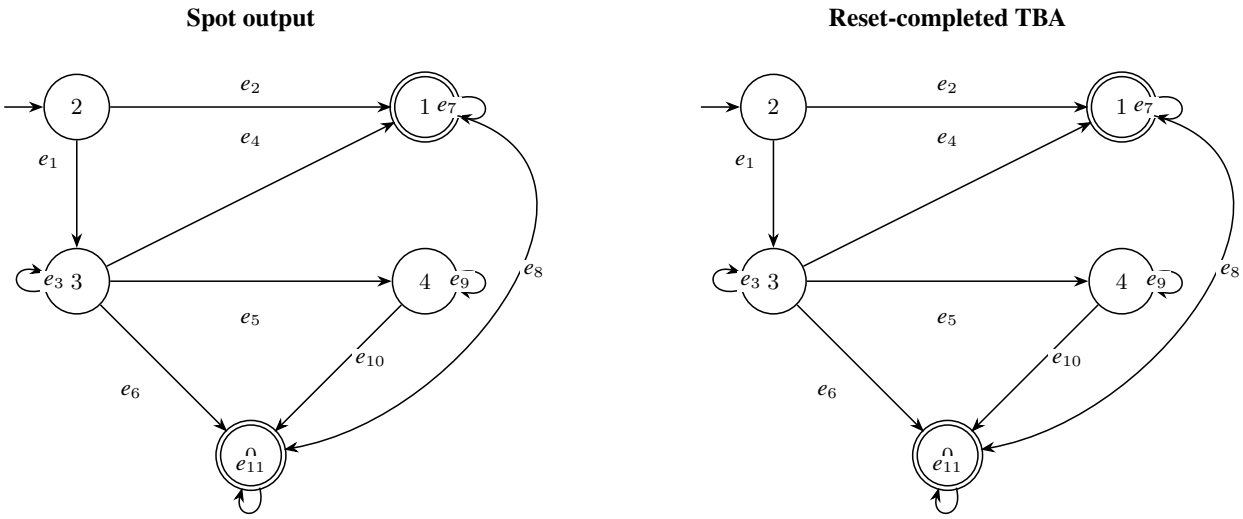


FIGURE 4. **The saved Spot 2.16 graph (redrawn, left) and its hand-derived reset-completed TBA (right; same state IDs). Edge identifiers are expanded in Table 3. This graph-to-TBA illustration is not the formally verified alarm-response backend instance.**

TABLE 3. **Edge key for Fig. 4. P = LAct(p), N = LNAct(p), n = ¬LAct(p), X = ($x \le 5$), Y = ($y \ge 2$), U = LUnch(x), and R = LRst(y). In the timed interpretation, P means event p and both N and n mean event $\neg p$. Negative Y literals are removed by relaxation.**

Edge	From	To	Spot propositional cube	Timed event; guard; reset set
e_1	2	3	$N \wedge n \wedge R$	$\neg p$; $\top$; $\{x, y\}$
e_2	2	1	$N \wedge P \wedge R$	$p \wedge \neg p$ (unrealizable); $\top$; $\{x, y\}$
e_3	3	3	$N \wedge n \wedge U \wedge X \wedge \neg Y$	$\neg p$; $x \le 5$; $\varnothing$
e_4	3	1	$N \wedge P \wedge X \wedge \neg Y$	$p \wedge \neg p$ (unrealizable); $x \le 5$; $\{x\}$
e_5	3	4	$n \wedge U \wedge X \wedge Y$	$\neg p$; $x \le 5 \wedge y \ge 2$; $\varnothing$
e_6	3	0	$P \wedge X \wedge Y$	p; $x \le 5 \wedge y \ge 2$; $\{x\}$
e_7	1	1	$N \wedge \neg Y$	$\neg p$; $\top$; $\{x\}$
e_8	1	0	Y	any event; $y \ge 2$; $\{x\}$
e_9	4	4	$n \wedge U \wedge X$	$\neg p$; $x \le 5$; $\varnothing$
e_{10}	4	0	$P \wedge X$	p; $x \le 5$; $\{x\}$
e_{11}	0	0	$\top$	any event; $\top$; $\{x\}$

clock was reset at i, its value at $i+1$ is at least Δ_i. Preservation after $i+1$ therefore gives

$$\mathsf{val}_\rho(j,x) \ge \Delta_i + \mathsf{elap}_w(i+1,j) = \mathsf{elap}_w(i,j).$$

The endpoint guard $x \le d$ implies $\mathsf{elap}_w(i,j) \le d$; the guard $x < d$ implies $\mathsf{elap}_w(i,j) < d$. Induction on P, Q yields the hatted Until witness and prefix in both cases. The strong Until is essential: an infinite continuation with no Q endpoint is not a bounded Until witness.

Lower-bounded hatted Until. Here the translation requires $\mathsf{rst}(x)$ at i. The finite branch of Γ waits for a position with $x \ge d$ (or $x > d$) and then supplies an ordinary $P \mathsf{U} Q$ witness. Since subsequent resets can only make a clock younger, $\mathsf{val}_\rho(j,x) \le \mathsf{elap}_w(i,j)$; thus the threshold guard proves the required lower-bound comparison. The nested Until supplies P up to the Q witness. The other branch is $\mathsf{G}P \wedge \mathsf{GF}Q$. It gives an unbroken P prefix and unboundedly many Q positions; divergence of timestamps lets us choose a Q witness beyond any finite lower bound. For the strict case, choose a recurring Q at distance at least d from $i+1$. Since $t_{i+1} > t_i$, that same event is strictly more than d after i. This is the additional argument in sound_UhatGt; it preserves the strict boundary without changing the constant.

Upper-bounded hatted Release. The translation resets the clock at i and uses weak Until with Q as its left condition. Consequently, Q holds at each future event until either $P \wedge Q$ discharges the Release obligation or the complementary clock test marks the end of the upper window. For a non-strict upper bound, the window ends at $x > d$; for a strict upper bound, it ends at $x \ge d$. If the clock has been reset again, its value is no larger than the elapsed time from i, so either endpoint guard can hold only after the corresponding physical-time window has ended. If weak Until ends at $P \wedge Q$, P at that earlier event discharges later Release obligations. This proves both comparisons with their exact strict boundary.

Lower-bounded hatted Release. Before the threshold, the left side of weak Until requires both the pre-threshold comparison and $\mathsf{unch}(x)$. A non-strict lower bound uses $x < d$; a strict lower bound uses $x \le d$. Thus an arbitrary reset trace cannot move the clock backwards while the pre-threshold condition remains active. Once the comparison becomes false, the translation requires ordinary $P \mathsf{R} Q$ from that event, unless the right-side alternative has already terminated

the obligation at a position satisfying P while still below the threshold. An older initial clock value can make the translation enter the ordinary Release branch earlier than the MTL interval does; this only imposes a stronger condition. The endpoint of the MTL lower interval is covered in both cases, including the event exactly at d for $\geq d$ and its exclusion for $> d$.

The standard LTL semantics used here are formalized in Rocq as LG_semantics, LGF_semantics, LW_semantics, and LF_semantics. Time divergence is used only for the fairness witness in lower-bounded Until; no canonical-reset assumption is used in any soundness family.

B. CANONICAL RESET POLICY AND RESIDUAL OBLIGATIONS

Completeness must choose one clock trace for each primitive formula, even when activations overlap. The canonical policy preserves a reference only while its residual obligation is still useful. Let $\mathsf{elap}_w(i,j) = t_j - t_i$. For upper-bounded Until define

$$\mathsf{URes}_{\leq}(i,v,d,p,q) \iff \tag{10}$$
$$\exists j \geq i.\ v + \mathsf{elap}_w(i,j) \leq d$$
$$\wedge\ w,j \models_{\mathrm{MTL}} q$$
$$\wedge\ \forall k \in [i,j).\ w,k \models_{\mathrm{MTL}} p, \tag{11}$$

$$\mathsf{URes}_{<}(i,v,d,p,q) \iff \tag{12}$$
$$\exists j \geq i.\ v + \mathsf{elap}_w(i,j) < d$$
$$\wedge\ w,j \models_{\mathrm{MTL}} q$$
$$\wedge\ \forall k \in [i,j).\ w,k \models_{\mathrm{MTL}} p. \tag{13}$$

The residual says that a q endpoint is still reachable with the current clock age and the required p prefix. For lower-bounded Release define

$$\mathsf{RRes}_{\geq}(i,v,d,p,q) \iff \tag{14}$$
$$\forall j \geq i.\ d \leq v + \mathsf{elap}_w(i,j) \Rightarrow$$
$$\Big(w,j \models_{\mathrm{MTL}} q \vee$$
$$\exists k \in [i,j).\ w,k \models_{\mathrm{MTL}} p\Big), \tag{15}$$

$$\mathsf{RRes}_{>}(i,v,d,p,q) \iff \tag{16}$$
$$\forall j \geq i.\ d < v + \mathsf{elap}_w(i,j) \Rightarrow$$
$$\Big(w,j \models_{\mathrm{MTL}} q \vee$$
$$\exists k \in [i,j).\ w,k \models_{\mathrm{MTL}} p\Big). \tag{17}$$

These are universal residuals: every position already beyond the threshold must have q or an earlier p that discharges the obligation. Their strict variants keep the comparison itself strict.

For a clock $x = F$, let $v = \mathsf{val}_{\rho^\star}(i,x)$. The policy is: The resulting values begin at $u_0 = 0$ and evolve by

$$u_{i+1} = \begin{cases} \Delta_i, & \text{if the policy resets at } i, \\ u_i + \Delta_i, & \text{otherwise.} \end{cases}$$

TABLE 4. Canonical reset decision at position i. A preservation row gives the condition under which the clock is not reset; otherwise it is reset.

Primitive formula F	Decision at i
$p\,\widehat{\mathsf{U}}_{\leq d}\,q$	Preserve iff $v \leq d$, $\mathsf{URes}_{\leq}(i,v,d,p,q)$, and $w,i \not\models_{\mathrm{MTL}} q$.
$p\,\widehat{\mathsf{U}}_{<d}\,q$	Preserve iff $v < d$, $\mathsf{URes}_{<}(i,v,d,p,q)$, and $w,i \not\models_{\mathrm{MTL}} q$.
$p\,\widehat{\mathsf{U}}_{\geq d}\,q, p\,\widehat{\mathsf{U}}_{>d}\,q, p\,\widehat{\mathsf{R}}_{\leq d}\,q,$ $p\,\widehat{\mathsf{R}}_{<d}\,q$	Reset iff $w,i \models_{\mathrm{MTL}} F$.
$p\,\widehat{\mathsf{R}}_{\geq d}\,q$	Preserve iff $v < d$, $\mathsf{RRes}_{\geq}(i,v,d,p,q)$, and $w,i \not\models_{\mathrm{MTL}} p$.
$p\,\widehat{\mathsf{R}}_{>d}\,q$	Preserve iff $v \leq d$, $\mathsf{RRes}_{>}(i,v,d,p,q)$, and $w,i \not\models_{\mathrm{MTL}} p$.

This is a word-dependent completeness witness, not a reset algorithm in an executable translator. Its decisions are keyed by the primitive formula itself, not by an occurrence path.

C. COMPLETENESS FOR ONE CANONICAL TRACE

For every satisfying formula, completeness constructs one clock-consistent extension $\rho^\star$ that serves all activations of every shared clock. The four operator families use different invariants; strict variants replace each boundary test by its strict counterpart.

Upper-bounded hatted Until. Choose a future q witness $j > i$ while the activation remains pending. The key invariant along the translated strong-Until prefix is

$$\mathsf{val}_{\rho^\star}(n,x) + \mathsf{elap}_w(n,j) \leq d \quad (i < n \leq j). \tag{18}$$

For $< d$, replace $\leq d$ by $< d$. The invariant need not hold at i: the translation begins with X, so the first clock test is at $i+1$. If the policy resets at i, then $u_{i+1} = \Delta_i$ and $u_{i+1} + \mathsf{elap}_w(i+1,j) = \mathsf{elap}_w(i,j)$, which is within the bound for the chosen witness. If it preserves at i, the residual supplies a still reachable q endpoint and p prefix; because the current q has not discharged that residual, its witness is strictly after i, and the same identity advances the residual inequality to $i+1$. At every intermediate event the policy either preserves this useful older reference or resets to a new reference whose MTL obligation is true. The endpoint satisfies q and the clock comparison. An older reference is at least as old as a newer one, so if it still reaches the shared endpoint within d, the shorter newer-prefix obligations are covered as well. This is the invariant proved by complete_UhatLe and complete_UhatLt.

Lower-bounded hatted Until. Whenever the formula is true at an activation, the policy resets its clock. Starting from $i+1$, a first position at which the canonical clock reaches d (or strictly exceeds d) and has a qualifying later q witness gives the finite branch of Γ: the clock guard supplies the lower-bound test, and the nested $P\,\mathsf{U}\,Q$ supplies the remaining witness. Before the first such position, the proof maintains an active-witness invariant. At each $n \geq i+1$ it supplies an activation $s < n$, a witness $m \geq n$ with $t_m - t_s \geq d$ and $w,m \models q$, p at every intermediate position, and $\mathsf{val}_{\rho^\star}(n,x) = t_n - t_s$.

Thus p holds at n and a q witness remains at or after every n. If a first qualifying position occurs, the finite branch applies; if none occurs, the invariant yields $\mathsf{G}P \wedge \mathsf{GF}Q$, the fairness branch of Γ. Divergence converts recurring q positions into a witness beyond the lower bound. For the strict case, replace $d \leq t_m - t_s$ in the invariant by $d < t_m - t_s$ and use threshold $x > d$; in the fairness argument, a recurring q at distance at least d from $i+1$ is strictly more than d from i because timestamps strictly increase. These are the arguments formalized by complete_UhatGe and complete_UhatGt.

Upper-bounded hatted Release. At every activation where the hatted Release formula is true, the policy resets the restart clock. The first future event must satisfy q, as required by the hatted Release semantics. The universal condition then continues as weak Until: q holds until either $p \wedge q$ discharges all later points or the clock passes the end of the bounded window. For a $\leq d$ window the stopping guard is $x > d$; for a $< d$ window it is $x \geq d$. Resetting at a later true activation can extend the clock window, but the later formula's truth supplies the corresponding future q-or-discharge condition. Thus a single reset trace satisfies both the old and new obligations. This proves complete_RhatLe and complete_RhatLt.

Lower-bounded hatted Release. While $\mathsf{RRes}_{\geq}$ holds, $v < d$, and p has not discharged the obligation, the policy preserves the clock. Preservation advances both the clock and the residual by

$$(i, v) \longmapsto (i{+}1, v{+}\Delta_i), \qquad \mathsf{elap}_w(i,j) = \Delta_i {+} \mathsf{elap}_w(i{+}1,j).$$

Substitution in (15) proves $\mathsf{RRes}_{\geq}(i+1, v+\Delta_i, d, p, q)$. Its strict analogue proves the same shift for $\mathsf{RRes}_{>}$ when $v \leq d$. The residual ensures that every event in the already-entered lower-bound region has q or an earlier p. If p occurs while the clock is still below threshold, the right side of weak Until terminates with that p; otherwise the clock reaches the threshold and the residual supplies the ordinary Release continuation. For $\geq d$, an event exactly at d is already in the constrained region; for $> d$, the prefix includes equality and only a strictly later event starts the region. The corresponding proofs are complete_RhatGe, rge_residual_shift, complete_RhatGt, and rgt_residual_shift.

D. OCCURRENCE-INDEXED STRUCTURAL INDUCTION

The completeness proof must recurse through a syntax tree while preserving the root formula's clock identities. Write $\mathsf{node}(\mathit{root}, \pi) = f$ when the path lookup at π in *root* returns f (the Rocq notation is `node_at root pi = Some f`). Let $\rho^\star = \mathsf{canonical_ext}(\mathit{root}, w)$. The strengthened theorem is:

Theorem V.1 (Canonical correctness at every syntactic occurrence)**.** *For every well-formed root formula, every path π with $\mathsf{node}(\mathit{root}, \pi) = f$, and every position i,*

$$w, i \models_{\mathrm{MTL}} f \iff \rho^\star, i \models_{\mathrm{LTL}} \mathcal{T}_\pi(f).$$

The same extension $\rho^\star$ is constructed once from root and w and is used for all paths.

Proof. Proceed by structural induction on f. Boolean and untimed temporal cases follow from their semantics and the induction hypotheses. For a binary constructor, node_at_left_binary and node_at_right_binary locate the child formulas at the extended paths. Ordinary timed operators unfold to the derived Boolean/hatted definitions, leaving the eight primitive cases proved above. In each hatted case, the forward direction uses its canonical-completeness lemma and the reverse direction its policy-independent soundness lemma. The path locates the formula for induction; it is not part of the clock key. The Rocq theorem is canonical_encoding_all. □

Clock identity is structural equality of the primitive formula value, including constructor, operands, comparison, and real bound. Hence if $\mathsf{node}(\mathit{root}, \pi_1) = F = \mathsf{node}(\mathit{root}, \pi_2)$ for a primitive timed formula, both paths refer to the same clock and the same canonical trace. Rocq proves this as identical_nodes_share_clock and identical_nodes_share_canonical_trace. The theorem therefore uses one extension for the entire root formula, not a separate witness per occurrence or activation.

E. CLOCKED-LTL CORRECTNESS AND TBA COMPOSITION

Theorem V.2 (Correctness of the clock encoding)**.** *For every well-formed formula f and timed word w,*

$$w, 0 \models_{\mathrm{MTL}} f \iff \exists \rho.\ \mathsf{same_base}(\rho, w) \wedge \mathsf{clock_consistent}(\rho) \wedge \rho, 0 \models_{\mathrm{LTL}} \mathcal{T}(f).$$

This is EncodingCorrect_proved. Its left-to-right direction uses the one canonical extension of Theorem V.1 at the root path. Its right-to-left direction uses the soundness induction for an arbitrary extension ρ; completeness cannot be used to reason about an arbitrary reset trace. Thus the quantifiers differ: every consistent satisfying extension is sound, whereas completeness constructs one extension.

Corollary V.3 (One clock per distinct primitive timed subformula)**.** *All occurrences of the same primitive timed formula use the same clock, and the encoding uses exactly the clocks in $\mathit{Clock}(f)$. The clock count is independent of the number of syntax-tree occurrences and dynamic activations.*

Proof. The translation is indexed by elements of $\mathit{Clock}(f)$, while T_uses_every_clock proves that each such clock occurs in $\mathcal{T}(f)$. The canonical trace of Theorem V.1 serves every activation of that formula. □

For a propositional Büchi automaton A over the extended atoms, let $\mathsf{compile_with}(A)$ denote relaxation followed by reset completion. Its correctness premise, Assumption IV.1, is language equivalence over *all* propositional words, including valuations that are not clock-consistent. The single end-to-end theorem is:

Theorem V.4 (Conditional MTL-to-TBA correctness)**.** *For every well-formed formula f, timed word w, and propositional Büchi automaton A satisfying Assumption IV.1,*

$$w, 0 \models_{\mathrm{MTL}} f \iff \mathsf{TBA_accepts}(\mathsf{compile_with}(A), w).$$

In Rocq this is MTL_to_TBA_correct_with; backend language equivalence is an explicit premise.

Proof. For MTL-to-TBA, the encoding theorem supplies a canonical clock-consistent extension satisfying $\mathcal{T}(f)$. The backend premise gives an accepting propositional run. Relaxation carries the run to the relaxed automaton by relax_complete; reset completion then supplies the completed clock-consistent extension and accepting TBA run by complete_complete.

For TBA-to-MTL, an accepting completed run first projects to a run of the relaxed backend by complete_sound. At each position, reconstruct a propositional valuation satisfying the selected original backend cube: deleted negative extended literals are made false, surviving literals are preserved, and deleted irrelevant atoms are chosen to satisfy that cube. The same state sequence remains accepting. The backend contract gives $\mathcal{T}(f)$ on the reconstructed word. The relaxed word agrees on action atoms and only adds truth to relevant extended atoms, so positivity of $\mathcal{T}(f)$ transfers satisfaction to the relaxed word, as formalized by relax_sound. Applying the soundness direction of Theorem V.2 gives $w, 0 \models_{\mathrm{MTL}} f$. □

The convenience theorem MTL_to_TBA_correct chooses its backend using the project axiom LTL_TO_BUCHI_CORRECT. It is not assumption-free. The theorem displayed above instead takes backend equivalence as an explicit argument. This distinction is visible in `proof/ASSUMPTIONS.txt`; the concrete alarm example below proves a contract for one Rocq backend record and does not rely on Spot being verified.

F. ROCQ RESULT MAP AND ASSURANCE BOUNDARY

Table 5 maps the article's results to their checked declarations and assumptions. The primitive rows collect the four soundness and four canonical-completeness families, each with its strict variant.

The source-module aliases in Table 5 are:

Core. `MTL_to_TBA_Shared_Clock_Derived_Strict_Direct_Core.v`

Proof. `EncodingCorrect_Shared_Clock_Derived_Strict_Direct_Proof.v`

Alarm. `OverlappingResponse_Example.v`

The audit lists standard library dependencies for classical real comparison decisions, excluded middle, functional extensionality, and descriptions. These are distinct from the project-specific backend axiom. The theorem MTL_to_TBA_correct_with has no LTL_TO_BUCHI_CORRECT dependency; its backend premise is explicit. The convenience theorem does use that axiom. The alarm-response backend contract itself is proved for the Rocq `PBuchi` record under the standard/classical assumptions in the audit.

The formal domain remains deliberately bounded: well-formed core formulas; real-valued mathematical parameters (not executable encodings of arbitrary reals); strictly increasing divergent timestamps; event-guarded automata without location invariants; and existential nonnegative initial clock values. The zero lower-bound and zero strict upper-bound reductions described in Section II-A are preprocessing, not mechanized results. Parsing, a concrete LTL backend implementation, Spot execution, graph import, automaton printing, and adaptation to zero-initialized clocks remain outside the theorem. Tools such as UPPAAL start clocks at zero; relating that convention to the existential initial clocks of the theorem requires a check that no clock is read before its first reset, which is left to follow-up work. The sizes and checks of Section VI are measurements, not mechanized results.

The occurrence-indexed induction, residual invariants, and separate relaxation/reset-completion obligations expose the proof dependencies behind the language-preservation result. The development is supplied as inspectable Rocq source; the successful source replay procedure and its limits are documented with the artifact.

G. ALARM-RESPONSE PROOF INSTANCE

Consider $f_{alarm} = \mathsf{G}(alarm \Rightarrow \Diamond_{\leq 3} handled)$, meaning every alarm must be handled within three time units. In proof/OverlappingResponse_Example.v, the formula is `MR MFalse (MOr (MNotAtom ALARM) (MUle 3 MTrue (MAtom HANDLED)))`, where `ALARM` and `HANDLED` are actions 1 and 2. Rocq checks that it is well formed and has one primitive timed subformula, $\widehat{\mathsf{U}}_{\leq 3}\, \top\, handled$, whose obligations share clock x. Abbreviate $\mathsf{LAct}(alarm)$ by A, $\mathsf{LNAct}(alarm)$ by N, $\mathsf{LAct}(handled)$ by H, $x \leq 3$ by C, and $\mathsf{LUnch}(x)$ by U. Spot receives the exact clocked-LTL formula

$$\mathsf{G}\big(N \vee H \vee \mathsf{X}((C \wedge U)\, \mathsf{U}\, (C \wedge H))\big).$$

Spot 2.16's input and output are saved in examples/spot/alarm_response_backend.ltl and examples/spot/alarm_response_backend.dot [12]. The generated graph has initial state 0, accepting set $\{0, 1\}$, and these transitions:

source	target	label
0	0	$H \vee N$
0	1	$\neg H \wedge \neg N$
1	0	$C \wedge H$
1	2	$C \wedge \neg H \wedge U$
2	0	$C \wedge H$
2	2	$C \wedge \neg H \wedge U$

The graph is manually transcribed as alarm_response_spot_backend; Rocq splits the disjunctive self-loop into two cubes. Lemma alarm_response_T_eq matches its input to the Rocq translation; alarm_response_spot_backend_contract proves that the transcribed automaton has exactly that LTL language on all propositional words. This discharges the backend premise for this instance. The regeneration script requires `ltl2tgba -version` to report Spot 2.16.

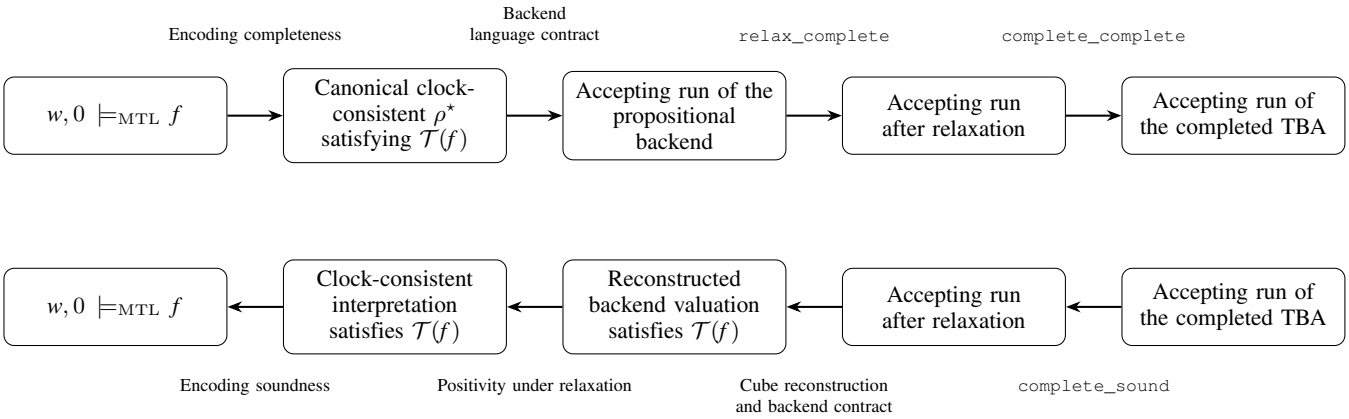


FIGURE 5. **Proof outline for Theorem V.4: completeness (top) and soundness (bottom), each using the backend equivalence contract.**

TABLE 5. **Mathematical results and Rocq assurance boundary.**

Mathematical result	Rocq declaration(s)	Source module	Key dependency
Derived ordinary-operator semantics	Eight strict/non-strict compatibility lemmas	Core	Well-formed real bounds and strictly increasing timed words
Canonical correctness at each path; one extension	canonical_encoding_all; EncodingCorrect_proved	Proof	Standard Rocq real/classical and function-extensionality assumptions
Primitive soundness and canonical completeness	sound_UhatLe/Ge/Lt/Gt, sound_RhatLe/Ge/Lt/Gt; corresponding complete_* lemmas	Proof	Soundness: any consistent resets; completeness: canonical policy
Relaxation and reset completion	relax_sound, relax_complete, complete_complete	Core	Positive extended atoms; label side conditions
Conditional backend composition	MTL_to_TBA_correct_with	Proof	Explicit all-propositional-words backend equivalence premise
Convenience backend theorem	MTL_to_TBA_correct	Proof	Project axiom plus standard Rocq assumptions
Alarm-response instance	alarm_response_spot_backend_contract; alarm_response_spot_tba_correct	Alarm	Contract proved for the Rocq record; Spot transcription is external

TABLE 6. **Timed transitions of the alarm-response Rocq record after relaxation and reset completion. The Graphviz DOT graph-to-record correspondence remains outside the Rocq proof.**

From	To	Event	Guard	Reset
0	0	H	$\top$	$\{x\}$
0	0	$\neg A$	$\top$	$\{x\}$
0	1	$A \wedge \neg H$	$\top$	$\{x\}$
1	0	H	$x \leq 3$	$\{x\}$
1	2	$\neg H$	$x \leq 3$	$\varnothing$
2	0	H	$x \leq 3$	$\{x\}$
2	2	$\neg H$	$x \leq 3$	$\varnothing$

The resulting concrete TBA is alarm_response_spot_tba. It has initial location 0, accepting locations $\{0, 1\}$, and clock x. Table 6 lists the reset-completed transitions in the Rocq record. Guards are checked before resets, and $\top$ denotes no clock guard. The theorem alarm_response_spot_tba_correct instantiates Theorem V.4 and proves that this record accepts exactly the timed words satisfying f_{alarm}. The verified object is the Rocq record and its contract; neither Spot nor the saved DOT-to-record transcription is verified.

Two trace instances make the universal theorem concrete. The accepted word has alarms at times 0 and $\frac{3}{2}$, a handled event at time 3, and only neutral actions at subsequent timestamps, which advance by $\frac{3}{2}$. Rocq proves alarm_overlap_trace_satisfies and derives alarm_overlap_trace_accepted_by_spot_tba. Both alarms use the same clock x. On the canonical extension, the residual checks at indices 0 and 1 are respectively

$$0 + (3 - 0) = 3 \leq 3, \qquad \tfrac{3}{2} + (3 - \tfrac{3}{2}) = 3 \leq 3.$$

Thus the clock is preserved at the second alarm: its existing value still leaves exactly enough time for the shared handler at time 3. The accepted-trace corollary follows from the checked MTL satisfaction proof and alarm_response_spot_tba_correct; it is not a separate graph search.

The rejected word has alarms at times 0 and 2, a handled event at time 4, and neutral actions thereafter at intervals of 2. The first request has no handler by time 3. Rocq proves alarm_response_trace_violates and derives alarm_response_trace_rejected_by_spot_tba. These are fixed trace instances of the semantic result, not performance experiments or validation of Spot's implementation.

The repository also provides a Python consistency check between the saved graph and Rocq record (up to state renaming, including guards), with mutation self-tests for a changed guard and accepting set. It is an executable audit of the transcription, not a verified importer or a proof of

Spot's semantics. The separate finite reset-policy scenario is illustrative.

VI. EVALUATION

The theorem says nothing about the size of the automaton, about the unverified components that produce it, or about how it compares with an independent implementation. This section measures the automaton compile(f) obtained by the translation $\mathcal{T}$, Spot, relaxation, and reset completion, before any further processing; no optimization or export is applied here. We ask three questions. Q1: how large is compile(f), compared with the automaton of CASAAL [11], which implements the construction of Li et al. [2]? Q2: on concrete formulas, do the languages agree with CASAAL's, as far as an independent timed-automaton checker, or independent evaluation on words, can tell? Q3: does the backend satisfy, on these formulas, the contract that the theorem assumes? The answers are measurements and tests; none of them is a mechanized result.

A. SETUP

The tool `mtl2tba` parses a formula, computes $\mathcal{T}(f)$, calls Spot 2.16 (`ltl2tgba`), reads its LBTT output, and applies relaxation and reset completion. The functions $\mathcal{T}$, relaxation, and reset completion are OCaml code extracted from the Rocq definitions; the parser, the call to Spot, the LBTT reader, the printers, and extraction itself are not verified. The tool is run with the options `-noweak -norecur`, so that the clocked-LTL formula is exactly $\mathcal{T}(f)$ as defined in Section III. Measurements were taken on an Intel Core i7-8650U (1.9 GHz, 11.7 GB) under Ubuntu 26.04 in WSL 2, with a limit of 600 s per tool run and per check (120 s for random formulas). The 31 configurations are fifteen specification patterns F1–F14 (F3b is F3 without hat), conjunctions Rk of k bounded responses $\square(a_i \Rightarrow \Diamond_{\leq 5} b_i)$ for $k = 1, \ldots, 8$, and nestings Nk of k bounded eventualities $\Diamond_{\leq 5}(p_1 \wedge \Diamond_{\leq 5}(p_2 \wedge \cdots))$. Scripts, inputs, CASAAL outputs, and raw results are in the artifact (`evaluation/`).

B. SIZE OF COMPILE(F) AND CASAAL REFERENCE

Table 7 gives, for each configuration, the reachable locations, transitions, and clocks of compile(f), and the size of CASAAL's output. CASAAL was run with mutually exclusive events, which matches our one-event-per-position words. The two counts of transitions are not directly comparable: CASAAL labels a transition with a Boolean formula over propositions, whereas Spot's output, and hence compile(f), has one transition per cube over the propositions and clock literals. In R4, for example, compile(f) has 48 locations and 1,625 transitions, and CASAAL 16 locations and 80 symbolic transitions; both use four clocks. Column Grp. T counts the transitions of compile(f) after grouping edges with the same source, target, guard, and resets into one edge labelled by the union of their events; it is a way of counting and changes no automaton. Grouping reduces R4 from 1,625 to 768 and N8 from 166 to 54 transitions, but a gap remains (CASAAL: 80 and 19), so the difference is not explained by the representation of labels alone.

TABLE 7. Automaton compile(f) before any optimization: locations/transitions/clocks (L/T/X), the transitions after grouping edges with the same source, target, guard, and resets into one (Grp. T; a count, not a transformation), CASAAL output with mutually exclusive events (locations/symbolic transitions/clocks; n/a: no exact CASAAL encoding), TChecker products P_1, P_2, S ("empty": all three empty), and the tested backend contract.

ID	L/T/X	Grp. T	CASAAL	P_1, P_2, S	Backend
F1	3/7/1	6	2/4/1	empty	yes
F2	2/5/1	5	1/1/0	empty	yes
F3	2/5/1	5	n/a	S only	yes
F3b	1/1/0	1	1/1/0	empty	yes
F4	3/7/1	6	2/4/1	empty	yes
F5	7/38/1	34	4/15/1	empty	yes
F6	7/38/1	34	4/15/1	empty	yes
F7	3/6/1	6	2/4/1	empty	yes
F8	5/15/1	15	2/4/1	empty	yes
F9	4/9/2	9	4/9/2	empty	yes
F10	5/13/3	12	5/12/3	empty	yes
F11	5/11/2	11	4/7/2	empty	yes
F12	5/23/2	23	3/9/2	empty	yes
F13	6/23/2	23	n/a	S only	yes
F14	8/38/2	29	8/18/2	empty	yes
R1	3/7/1	6	2/4/1	empty	yes
R2	8/45/2	32	4/12/2	empty	yes
R3	20/275/3	160	8/32/3	empty	yes
R4	48/1625/4	768	16/80/4	timeout	yes
R5	112/9375/5	3584	32/192/5	timeout	yes
R6	timeout	–	64/448/6	–	yes
R7	timeout	–	128/1024/7	–	check timeout
R8	timeout	–	256/2304/8	–	tool timeout
N1	3/5/1	5	3/5/1	empty	yes
N2	4/11/2	9	4/7/2	empty	yes
N3	5/21/3	14	5/9/3	empty	yes
N4	6/36/4	20	6/11/4	empty	yes
N5	7/57/5	27	7/13/5	timeout	yes
N6	8/85/6	35	8/15/6	timeout	yes
N7	9/121/7	44	9/17/7	timeout	yes
N8	10/166/8	54	10/19/8	timeout	ltl2ba timeout

The number of clocks coincides with CASAAL's on every configuration that CASAAL encodes exactly, except F2: under mutual exclusion, $\square(p \Rightarrow \square_{\leq 3} q)$ forbids p altogether, and CASAAL produces a one-state automaton without clock, whereas compile(f) keeps two locations and one clock. This agrees with the clock discipline of the construction: one clock per structurally distinct primitive timed subformula. Locations and transitions are larger than CASAAL's on most configurations, sometimes by a large factor (R4, R5); the nesting family N has the same number of locations as CASAAL. The growth in the R family comes from Spot's automaton for the conjunction of k clocked-LTL clauses, before relaxation; R6–R8 exceed the 600-second limit. Construction time is below one second for every completed configuration except R4 (3.4 s), N6 (1.6 s), and R5 (43 s). These numbers describe the unoptimized output of the verified construction; they are not a comparison of tools; reducing these sizes by post-processing is left to follow-up work.

C. LANGUAGE CHECKS WITH TCHECKER

For a formula φ, let O and O' be compile(φ) and compile($\neg\varphi$), and C, C' CASAAL's automata for φ and $\neg\varphi$. TChecker 0.8 [13] decides the emptiness of the products $P_1 = O \times C'$, $P_2 = C \times O'$, and $S = O \times O'$, built over

infinite words with one event per position, strictly increasing and divergent timestamps, and clocks starting at 0. P_1 and P_2 empty mean that O is disjoint from C' and C from O'; if CASAAL's two automata are complementary, which is not checked, this gives $L(C) \subseteq L(O)$ and $L(C') \subseteq L(O')$. The checks therefore establish inclusions, not equality. S empty means that our automata for φ and $\neg\varphi$ never accept the same word. A fourth product $O \times C$ is a sanity check: it must be non-empty when φ is satisfiable.

On the benchmark configurations, P_1, P_2, and S are empty for every configuration that completes within the limits (F1–F14 except the hatted F3 and F13, which CASAAL cannot express, R1–R3, and N1–N4); for F3 and F13 only S is checked, and it is empty. For R5 and N6 the three checks exceed 600 s. At 1,800 s, run singly, P_2 of R4 is empty, while P_1, S of R4 and all three checks of N5 still time out (which may also mean memory exhaustion). R6–R8 produce no automaton within the limit. For N7 and N8, O is built (26 s and 101 s), but P_1 exceeds 600 s and O' is not built within the limit, so P_2 and S are not run (inconclusive). The sanity product is non-empty in every completed case.

The artifact also contains 100 random formulas of depth at most three over three propositions, generated with their CASAAL automata. For 99 of them, P_1, P_2, and S are empty, with no timeout; for five of these, the sanity product is empty as well, and inspection shows that each of these five formulas is unsatisfiable under mutual exclusion (for example $q \wedge \Diamond_{>2} p \wedge r$). The remaining formula, X74, is $\Diamond(q \wedge r) \Rightarrow egp$, for which P_1 is non-empty. Its negation $\Diamond(q \wedge r) \wedge p$ is unsatisfiable under mutual exclusion, and our automaton for it has no accepting location. CASAAL's printed automaton for the negation has no acceptance mark, and the conversion script reads an automaton without marks as accepting every run (as for $\mathsf{F}q \wedge \mathsf{G}\neg q$). We report X74 as inconclusive: it reflects how acceptance is printed and read, not a disagreement that the products can establish.

D. AGREEMENT ON ULTIMATELY PERIODIC WORDS

The TChecker checks establish inclusions and time out on the larger configurations, so we add a test of equality on finitely many words. For each configuration we draw random ultimately periodic timed words uv^ω (a prefix of 0–4 and a loop of 1–5 events, integer timestamps in half units, strictly increasing and divergent; half of the words follow the order of the propositions of the formula), and compare three verdicts: the formula under the pointwise semantics above, evaluated by an independent Python implementation; acceptance by $\mathsf{compile}(f)$; and acceptance by CASAAL's automaton. Clocks of CASAAL start at 0; for $\mathsf{compile}(f)$ initial clock values are tried on a grid when it has at most three clocks, since the theorem quantifies them existentially. On the 28 benchmark configurations that build (400 words each, 11,200 in all, 4,495 satisfying the formula) and on the 100 random formulas (150 words each, 15,000 in all, 8,273 satisfying it), the semantics and $\mathsf{compile}(f)$ agree on every word. CASAAL is included on 26 of the 28 benchmark configurations (not on the hatted F3 and F13) and on all random formulas, and agrees on every word there as well: 10,400 plus 15,000 words, 25,400 three-way comparisons; the other 800 words compare only the semantics and the automaton. As a negative control, the same checker finds disagreements for F1 against the formulas with bound 4 instead of 5 (11 of 300 words) and with a strict bound (5 of 300). The test has little weight for some formulas: no word of N8 satisfies the formula among the random ones, so for N5–N8 we added five hand-built words each (accepted and rejected; all agree), and nine random formulas have fewer than five satisfying and fourteen fewer than five violating words. The words are finitely many and short; this is a test, not a proof of equality.

E. BACKEND CONTRACT

Theorem MTL_to_TBA_correct_with assumes that the propositional automaton accepts exactly the models of $\mathcal{T}(f)$. For each configuration we saved the formula given to Spot and the LBTT automaton A it returned, and checked the contract with Spot's `autfilt`, using an independent translator, ltl2ba 1.3: $A \times \mathrm{ltl2ba}(\neg\mathcal{T}(f))$ and $\overline{A} \times \mathrm{ltl2ba}(\mathcal{T}(f))$ must both be empty. The contract holds on 28 of the 31 configurations; for R7 the check exceeds 600 s, for N8 ltl2ba exceeds 300 s, and for R8 Spot does not finish. On every configuration where Spot produced an automaton, the number of states and transitions read by the LBTT reader of `mtl2tba` equals the number reported by `autfilt` on the same file. These checks test the contract on these inputs; they do not prove it, and they use Spot's own tools for part of the check.

F. HATTED OPERATORS

F3 and F13 use the hatted operators of Section II. CASAAL has no hatted operator; its closest encoding of F3, $\Box(e \Rightarrow \mathsf{X}\Box_{\leq 2}\neg e)$, measures the window from the next event rather than from the triggering one (Fig. 2) and is not equivalent, so we do not compare the two languages. The time reference is visible in the automata. Under mutual exclusion the non-strict window of the unhatted F3b contains the triggering event, so F3b forbids e altogether: $\mathsf{compile}(f)$ has one location, one transition, and no clock. The window of F3 starts after the triggering event but is measured from its timestamp, and $\mathsf{compile}(f)$ has two locations, five transitions, and one clock x, reset on each e; after an e, a further e is accepted only with the guard $x > 2$.

G. THREATS TO VALIDITY

The configurations and random formulas are small and of our choosing; the R family shows that the size of Spot's automaton grows quickly with the number of conjuncts. The CASAAL automata were produced earlier and not regenerated; CASAAL's integer bounds, nondecreasing time, and acceptance printing differ from our model, and the inclusions depend on the unchecked complementarity of its automata. The TChecker products, the word-level test, and the conversion scripts are unverified code. Timeouts are reported as inconclusive.

VII. ARTIFACT AND REPRODUCIBILITY

The artifact is available at https://github.com/efares1/rocq-mtl-to-tba-translation; the revision described here is the release tagged `v1.0`. It contains the Rocq sources, the assumptions audit, the tool `mtl2tba`, the evaluation scripts with their inputs and raw results, the Spot input and graph of the alarm-response example, and the article source. The proofs build with Rocq 9.0.1: `make` in `proof/` compiles the core, the correctness development, and the alarm example; `proof/CheckAssumptions.v` prints the assumptions of the main theorems, which are recorded in `proof/ASSUMPTIONS.txt`; and `coqchk` re-checks the compiled objects. On Windows, `reproduce-ieee-access.ps1` performs these steps, runs the two Python assurance scripts, and builds the PDF.

The main checked claims have direct source declarations: EncodingCorrect_proved and canonical_encoding_all establish the formula-to-clocked-LTL equivalence; MTL_to_TBA_correct_with composes this result with the explicit all-propositional-words backend contract; alarm_response_spot_tba_correct is the concrete alarm instance. The primitive theorem families, relaxation lemmas, reset-completion lemmas, and their assumptions are mapped in Table 5.

The evaluation of Section VI is reproduced, under Linux or WSL with OCaml, Spot 2.16, TChecker 0.8, and ltl2ba 1.3, by building the tool in `tool/` and running `evaluation/run_compiled_eval.sh` and `evaluation/run_spot_validation.sh`; `evaluation/make_compiled_table.py` regenerates Table 7 from the raw results. The word-level test is `evaluation/lasso_check.py`, run on the automata written by `evaluation/dump_compile_dot.sh`, and `evaluation/grouped_edges.py` computes the grouped transition counts. The SHA-256 digests of the Rocq core and correctness files are listed in `SHARED_FOUNDATION_MANIFEST.txt`, and a script checks them, so that later work can reuse exactly this foundation.

VIII. RELATED WORK

Pointwise MTL has extensive decidability and expressiveness results [8]–[10], [14], [15]. Unrestricted dense-time MTL has no general effective translation to finite ordinary timed automata; bounded-variability and alternating-automaton results have different scopes [16], [17]. $\mathrm{MTL}_{0,\infty}$ retains common response and separation properties [1], [15]. Our hatted operators make the timing reference explicit in the logic and proof.

Translations of timed logics. Li et al. establish the full-fragment mathematical language-equivalence result. Our overlapping-activation proof checks completeness for the present formal model; we do not infer that prior constructions lacked such handling.

Xu and Miao describe a PVS-based transformation from MITL formulas to fair timed automata, including construction principles and implementation details [3]. This is an earlier proof-assistant-based timed-logic translation; our scope-specific comparison concerns the pointwise $\mathrm{MTL}_{0,\infty}$ construction formalized here.

MITL translations use temporal testers, automata networks, or one-clock alternating automata [17]–[19]; MightyL and MITL2Timed provide UPPAAL-oriented tools for that distinct logic [20], [21]. Obligation-merging techniques are related, while our one-sided residual predicates support the checked completeness proof.

Clocked temporal logic and subformula clocks. The clocked linear temporal logic extension CLTLoc adds clock comparisons and reset predicates and has been used to encode metric logics [22]. Our similar propositional vocabulary is an automaton-synthesis intermediate: the backend treats clock tests and reset markers as propositions. Positivity gives upward closure, and reset completion restores timed meaning; hence the explicit relaxation proof.

Event-clock logics associate clocks with subformulas to measure time since or until events [16], [23], [24]. Formula-indexed clocks are established; our proof checks that one clock per distinct timed formula preserves language across repeated activations and all strict and non-strict cases (Section V-D).

Recent automata-based real-time verification. Recent neighboring work addresses different settings: an improved one-clock alternating MTL automaton for finite timed words with clock deactivation and zone analysis [25]; and TEMPORA's pointwise MITL model checking with past and future modalities [26]. These do not translate our $\mathrm{MTL}_{0,\infty}$ fragment to infinite-word TBAs. Ho et al.'s MightyPPL translates pointwise MITL with past and Pnueli modalities over finite and infinite timed words; it gives mathematical construction results and proof sketches alongside a complete implementation whose testers sequentialize overlapping obligations [27]. The September 2026 QEST+FORMATS preprint describes an implementation for existentially punctual metric temporal logic with past and Pnueli modalities (MTLPPL) [28]. Its term *one-sided MTL* denotes the subfragment without past or Pnueli, whereas here it describes interval shapes $[0, d]$ and $[d, \infty)$. Any syntactic containment comparison is limited to conventional source formulas after zero-bound reduction and with integer bounds; it excludes hatted abstract syntax tree (AST) constructors. The QEST fragment also permits punctual and non-one-sided intervals in its stated positions. Its punctual-response example does not imply support for unrestricted punctual MTL; differing bounds and timed-word conventions preclude a shared-input semantic comparison.

These tools address different fragments or output models; this paper neither re-verifies them nor makes a shared-input, coverage, clock-count, or performance comparison. The Rocq development is not integrated with CASAAL, MightyPPL, or another checker.

Mechanized translations and trusted backends. Proof-assistant work already verifies LTL-to-Büchi constructions and model checkers [5], [29]–[31]. Wang et al. verify an MLTL-to-regular-expression transformation in Isabelle/HOL, export executable code, and compare it with

TABLE 8. Representative mathematical results and mechanization evidence for timed-logic-to-automata constructions. The rows have different logics, semantics, targets, and proof scopes; they are not a performance comparison.

Work	Semantic scope	Existing correctness and construction	Mechanization and implementation evidence
Bulychev [1]	Pointwise non-Zeno infinite words; integer bounds, all four comparisons, nondecreasing timestamps.	Theorems 1–2 prove exact automaton languages for safety/co-safety. The construction already uses timed-subformula clocks, obligations, and reset/unchanged markers.	Mathematical proofs and controller-synthesis tooling for the stated fragments.
Li et al. [2]	Full $\mathrm{MTL}_{0,\infty}$; pointwise non-Zeno words, integer bounds, all four comparisons, nondecreasing timestamps, zero initial clocks.	Establishes exact language equality for the timed-Büchi construction and language preservation under best-choice resets.	CASAAL implements the workflow and deterministic approximations [11]; the proof is mathematical.
Xu and Miao [3]	MITL formulas mapped to fair timed automata; exact semantic relation to this paper's pointwise event-word model is not compared.	The article describes a syntax/semantics-based construction by formula type.	The publisher abstract reports PVS implementation details; detailed proof coverage is not assessed in this comparison.
This paper	Well-formed Rocq formulas with real bounds, strictly increasing timestamps, existential nonnegative initial clocks.	EncodingCorrect_proved proves clocked-LTL equivalence for all eight primitive cases; TBA composition is conditional on backend equivalence.	Rocq-checked proof and one proved contract for a manual Spot transcription; tool evaluation with extracted code (Section VI).

existing implementations [6]. Chattopadhyay and Mamouras formalize in Coq a compositional construction of online monitors for past-time MTL over a discrete temporal domain, using quantitative semantics and Moore machines; they also extract OCaml monitors. This is a distinct monitoring target and semantic setting from the infinite-word TBA construction studied here [7]. For metric logics, Roohi and Viswanathan use PVS to correct an unsound release semantics in prior MITL decision procedures and prove supporting results for an adapted timed-automata construction under continuous-time signal semantics [4]. Xu and Miao's PVS-based construction is described above. Together, these works establish substantial proof-assistant precedents for related metric logics, monitors, and automata; this paper adds a Rocq-checked correctness development for the specified pointwise infinite-word TBA construction.

Event-B refinement patterns. Our work on Event-B patterns addresses their generation and refinement, including domain-specific language (DSL)-based pattern generation [32], patterns using weakest-precondition calculus [33], and a forthcoming 2027 article on pattern-based development using predicate transformers [34]. These works provide the Event-B pattern-generation context for our broader objective: given an MTL property, use its TBA translation as the input to a downstream Event-B pattern-generation step. The present paper establishes the MTL-to-TBA stage only. It gives no TBA-to-Event-B transformation or correctness result for generated Event-B patterns.

IX. CONCLUSION

This paper provides a Rocq-checked soundness-and-completeness development for the specified pointwise $\mathrm{MTL}_{0,\infty}$-to-TBA construction. Its proof covers all eight strict and non-strict primitive cases, soundness for arbitrary clock-consistent reset traces, and completeness using one canonical trace across repeated activations of formula-keyed clocks. The occurrence-indexed induction connects each syntactic occurrence to its translation while preserving the identity of a shared clock. This supplies a checked account of the obligations that a mathematical argument must discharge when several timed requirements overlap.

Rocq also checks relaxation and reset completion and composes them with an explicit propositional backend language-equivalence contract. The resulting theorem makes its backend dependency inspectable instead of hiding it in an assurance claim. This work builds on the mathematical language-preservation results of Li et al. and adjacent proof-assistant work, while contributing a machine-checked derivation for the formal construction and semantic model defined here [2]–[4].

For the alarm-response instance, the backend contract is proved for a manual Rocq transcription of a Spot graph. Rocq proves both acceptance of the overlapping-request trace, whose response satisfies the concurrent obligations, and rejection of a fixed violating trace. These examples make the theorem concrete; the external graph-to-record path remains outside its proof. The theorem applies to the stated pointwise semantics and composes through an explicit backend contract; Spot's graph generation/import and zero-bound preprocessing remain outside the checked boundary. Future work can extend the mechanization to these components and connect the semantic theorem to a verified executable backend.

The proof artifact and assumptions audit are described in Section VII.

REFERENCES

[1] P. Bulychev, A. David, K. Larsen, and G. Li, "Efficient controller synthesis for a fragment of MTL(0,∞)," *Acta Informatica*, vol. 51, no. 3-4, pp. 165–192, Jan. 2014.

[2] G. Li, P. G. Jensen, K. G. Larsen, A. Legay, and D. B. Poulsen, "Practical controller synthesis for $\mathrm{MTL}_{0,\infty}$," in *Proceedings of the 24th ACM SIG-*

SOFT International SPIN Symposium on Model Checking of Software, ser. SPIN 2017. New York, NY, USA: Association for Computing Machinery, 2017, pp. 102–111.

[3] Q.-G. Xu and H.-K. Miao, "Timed automata for metric interval temporal logic formulae in prototype verification system," *Journal of Shanghai University (English Edition)*, vol. 12, no. 4, pp. 339–346, 2008.

[4] N. Roohi and M. Viswanathan, "Revisiting MITL to fix decision procedures," in *Verification, Model Checking, and Abstract Interpretation (VMCAI 2018)*, ser. Lecture Notes in Computer Science, vol. 10747. Springer, 2018, pp. 474–494.

[5] A. Schimpf, S. Merz, and J.-G. Smaus, "Construction of Büchi automata for LTL model checking verified in Isabelle/HOL," in *Theorem Proving in Higher Order Logics (TPHOLs 2009)*, ser. Lecture Notes in Computer Science, vol. 5674. Springer, 2009, pp. 424–439.

[6] Z. Wang, K. Kosaian, and K. Y. Rozier, "Formally verifying a transformation from MLTL formulas to regular expressions," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2025)*, ser. Lecture Notes in Computer Science, vol. 15696. Springer, 2025, pp. 254–275.

[7] A. Chattopadhyay and K. Mamouras, "A verified online monitor for metric temporal logic with quantitative semantics," in *Runtime Verification (RV 2020)*, ser. Lecture Notes in Computer Science, vol. 12399. Springer, 2020, pp. 383–403.

[8] D. D'Souza and P. Prabhakar, "On the expressiveness of MTL in the pointwise and continuous semantics," *International Journal on Software Tools for Technology Transfer*, vol. 9, no. 1, pp. 1–4, 2007.

[9] J. Ouaknine and J. Worrell, "On the decidability of metric temporal logic," in *20th Annual IEEE Symposium on Logic in Computer Science (LICS 2005)*. IEEE Computer Society, 2005, pp. 188–197.

[10] P. Bouyer, F. Chevalier, and N. Markey, "On the expressiveness of TPTL and MTL," in *Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2005)*, ser. Lecture Notes in Computer Science, vol. 3821. Springer, 2005, pp. 432–443.

[11] G. Li, "CASAAL: A tool for translating $\mathrm{MTL}_{0,\infty}$ into timed Büchi automata," Online, 2017, available at the CASAAL tool page. [Online]. Available: https://lcs.ios.ac.cn/~ligy/tools/CASAAL/

[12] A. Duret-Lutz, A. Lewkowicz, A. Fauchille, T. Michaud, E. Renault, and L. Xu, "Spot 2.0 — a framework for LTL and ω-automata manipulation," in *Proceedings of the 14th International Symposium on Automated Technology for Verification and Analysis (ATVA'16)*, ser. Lecture Notes in Computer Science, vol. 9938. Springer, Oct. 2016, pp. 122–129.

[13] F. Herbreteau and G. Point, "TChecker: model checker for timed automata," https://github.com/ticktac-project/tchecker, 2025, version 0.8.

[14] R. Koymans, "Specifying real-time properties with metric temporal logic," *Real Time Syst.*, vol. 2, no. 4, pp. 255–299, 1990.

[15] R. Alur, T. Feder, and T. A. Henzinger, "The benefits of relaxing punctuality," *Journal of the ACM (JACM)*, vol. 43, no. 1, pp. 116–146, 1996.

[16] D. Ničković and N. Piterman, "From MTL to deterministic timed automata," in *Formal Modeling and Analysis of Timed Systems (FORMATS 2010)*, ser. Lecture Notes in Computer Science, vol. 6246. Springer, 2010, pp. 152–167.

[17] T. Brihaye, M. Estiévenart, and G. Geeraerts, "On MITL and alternating timed automata over infinite words," 2014. [Online]. Available: https://arxiv.org/abs/1406.4395

[18] O. Maler, D. Nickovic, and A. Pnueli, "From MITL to timed automata," in *Formal Modeling and Analysis of Timed Systems (FORMATS 2006)*, ser. Lecture Notes in Computer Science, vol. 4202. Springer, 2006, pp. 274–289.

[19] T. Ferrère, O. Maler, D. Ničković, and A. Pnueli, "From real-time logic to timed automata," *Journal of the ACM*, vol. 66, no. 3, 2019, art. no. 19.

[20] T. Brihaye, G. Geeraerts, H.-M. Ho, and B. Monmege, "MightyL: A compositional translation from MITL to timed automata," in *Computer Aided Verification*, R. Majumdar and V. Kunčak, Eds. Cham: Springer International Publishing, 2017, pp. 421–440.

[21] "MITL2Timed: Modifying Oddoux and Gastin LTL2BA to timed automata generation," GitHub repository. [Online]. Available: https://github.com/yzh89/MITL2Timed

[22] M. M. Bersani, M. Rossi, and P. San Pietro, "A tool for deciding the satisfiability of continuous-time metric temporal logic," *Acta Informatica*, vol. 53, no. 2, pp. 171–206, 2016.

[23] R. Alur, L. Fix, and T. A. Henzinger, "Event-clock automata: A determinizable class of timed automata," *Theoretical Computer Science*, vol. 211, no. 1–2, pp. 253–273, 1999.

[24] J.-F. Raskin and P.-Y. Schobbens, "The logic of event clocks: Decidability, complexity and expressiveness," *Journal of Automata, Languages and Combinatorics*, vol. 4, no. 3, pp. 247–286, 1999.

[25] P. Bouyer, B. Srivathsan, and V. Vishwanath, "Model-checking real-time systems: Revisiting the alternating automaton route," in *Foundations of Software Science and Computation Structures (FoSSaCS 2025)*, ser. Lecture Notes in Computer Science, vol. 15691. Springer Nature Switzerland, 2025, pp. 399–421.

[26] S. Akshay, P. Contractor, P. Gastin, R. Govind, and B. Srivathsan, "TEMPORA: Efficient verification of metric temporal properties with past in pointwise semantics," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*, ser. Lecture Notes in Computer Science, vol. 16505. Springer Nature Switzerland, 2026, pp. 640–659.

[27] H.-M. Ho, S. N. Krishna, K. Madnani, R. Majumdar, and P. Pandya, "MightyPPL: Model checking MITL with past and Pnueli modalities," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*, ser. Lecture Notes in Computer Science, vol. 16505. Springer Nature Switzerland, 2026, pp. 457–479.

[28] ——, "MightyPPL: Towards model checking MTL," 2026. [Online]. Available: https://arxiv.org/abs/2609.19073

[29] J. Esparza, P. Lammich, R. Neumann, T. Nipkow, A. Schimpf, and J.-G. Smaus, "A fully verified executable LTL model checker," in *Computer Aided Verification (CAV 2013)*, ser. Lecture Notes in Computer Science, vol. 8044. Springer, 2013, pp. 463–478.

[30] A. Schimpf and P. Lammich, "Converting linear-time temporal logic to generalized Büchi automata," *Archive of Formal Proofs*, 2014, https://isa-afp.org/entries/LTL_to_GBA.html, formal proof development.

[31] B. Seidl and S. Sickert, "A compositional and unified translation of LTL into ω-automata," Archive of Formal Proofs, https://isa-afp.org/entries/LTL_Master_Theorem.html, 2019.

[32] E. Fares, J.-P. Bodeveix, and M. Filali, "Pattern-based refinement generation through domain specific languages," in *Rigorous State-Based Methods (ABZ 2023)*, ser. Lecture Notes in Computer Science, vol. 14010. Springer, 2023, pp. 35–42.

[33] ——, "Correct pattern-based development through refinements and weakest preconditions calculus," in *Formal Aspects of Component Software (FACS 2024)*, ser. Lecture Notes in Computer Science, vol. 15189. Springer, 2024, pp. 59–78.

[34] ——, "Correct pattern-based development through refinements and predicate transformers," *Science of Computer Programming*, vol. 255, p. 103550, 2027, forthcoming; scheduled for January 2027.

ELIE FARES Elie Fares is with the Higher Colleges of Technology, Sharjah, UAE. His research interests include formal methods and mechanized verification.

JEAN-PAUL BODEVEIX Jean-Paul Bodeveix is with IRIT, Université de Toulouse, France. His research interests include formal methods and proof assistants.

HUSSAIN AL-AQRABI Hussain Al-Aqrabi is with the Higher Colleges of Technology, Sharjah, UAE. His research interests include cybersecurity and cloud security.

AZAR SALAMI Azar Salami is with the Higher Colleges of Technology, Sharjah, UAE. His research interests include formal methods.